

HR SQL Capstone Project Sample Query Results

Query 01

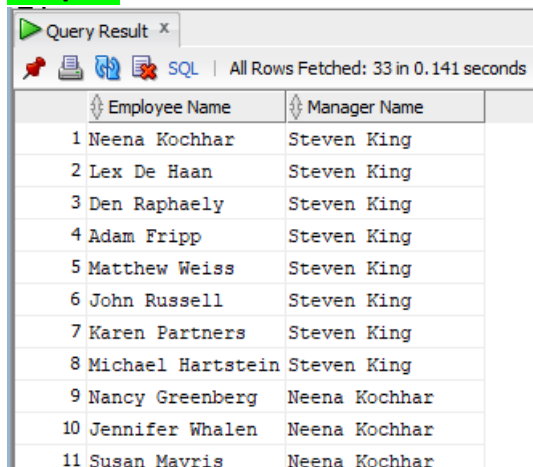
Question:

Show employees and their manager names using a self-join.

SQL Query:

```
SELECT e.first_name || ' ' || e.last_name AS "Employee Name",  
       m.first_name || ' ' || m.last_name AS "Manager Name"  
FROM employees e  
LEFT JOIN employees m  
ON e.manager_id = m.employee_id;
```

Output:



Employee Name	Manager Name
1 Neena Kochhar	Steven King
2 Lex De Haan	Steven King
3 Den Raphaely	Steven King
4 Adam Fripp	Steven King
5 Matthew Weiss	Steven King
6 John Russell	Steven King
7 Karen Partners	Steven King
8 Michael Hartstein	Steven King
9 Nancy Greenberg	Neena Kochhar
10 Jennifer Whalen	Neena Kochhar
11 Susan Mavris	Neena Kochhar

Concepts Used:

Self Join • LEFT JOIN • Table Aliases • String Concatenation

Query 02

Question:

Find employees earning more than their manager.

SQL Query:

```
SELECT e.first_name AS "Employee Name",  
       e.salary AS "Employee Salary",  
       m.salary AS "Manager Salary"  
FROM employees e  
JOIN employees m  
ON e.manager_id = m.employee_id  
WHERE e.salary > m.salary;
```

Output:

Query Result x		
SQL All Rows Fetched: 2 in 0.125 seconds		
	Employee Name	Employee Salary
1	Aarav	9500
2	Isha	9800

Concepts Used:

Self Join • JOIN • Comparison Operators • Filtering

Query 03

Question:

Show employees with their job grade based on their salary range.

SQL Query:

```
SELECT e.last_name,
       e.salary,
       g.grade_level
FROM employees e
JOIN job_grades g
ON e.salary BETWEEN g.lowest_sal AND g.highest_sal;
```

Output:

Query Result x		
SQL All Rows Fetched: 33 in 0.031 seconds		
	LAST_NAME	SALARY
1	Mikkilineni	2700 A
2	Baida	2900 A
3	Khoo	3100 B
4	Nayer	3200 B
5	Lorentz	4200 B
6	Whalen	4400 B
7	Pataballa	4800 B
8	Austin	4800 B
9	Fay	6000 C
10	Ernst	6000 C
11	Mavris	6500 C

Concepts Used:

JOIN • BETWEEN • Range Matching • Business Rules

Query 04

Question:

Show employees earning above their respective department average salary.

SQL Query:

```

SELECT e.*
FROM employees e
WHERE e.salary > (SELECT AVG(salary) FROM employees e2
WHERE e2.department_id = e.department_id);

```

Output:

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	100	Steven	King	SKING	515.123.4567	17-JUN-03	AD_PRES	24000	(null)	(null)	90
2	103	Alexander	Hunold	AHUNOLD	590.423.4567	03-JAN-06	IT_PROG	9000	(null)	102	60
3	104	Bruce	Ernst	BERNST	590.423.4568	21-MAY-07	IT_PROG	6000	(null)	103	60
4	108	Nancy	Greenberg	NGREENBE	515.124.4569	17-AUG-02	FI_MGR	12008	(null)	101	100
5	114	Den	Raphaely	DRAPHEAL	515.127.4561	07-DEC-02	PU_MAN	11000	(null)	100	30
6	121	Adam	Fripp	AFRIPP	650.123.2234	10-APR-05	ST_MAN	8200	(null)	100	50
7	120	Matthew	Weiss	MWEISS	650.123.1234	18-JUL-04	ST_MAN	8000	(null)	100	50
8	145	John	Russell	JRUSSEL	011.44.1344.429268	01-OCT-04	SA_MAN	14000	0.4	100	80
9	146	Karen	Partners	KPARTNER	011.44.1344.467268	05-JAN-05	SA_MAN	13500	0.3	100	80
10	201	Michael	Hartstein	MHARTSTE	515.123.5555	17-FEB-04	MK_MGR	13000	(null)	100	20
11	205	Shelley	Higgins	SHIGGINS	515.123.8080	07-JUN-02	AC_MGR	12008	(null)	101	110

Concepts Used:

Correlated Subquery • AVG() • Subquery • Department-level Analysis

Query 05

Question:

Show the top 3 highest-paid employees in each department.

SQL Query:

```

SELECT *
FROM (SELECT e.*, DENSE_RANK() OVER (
PARTITION BY department_id
ORDER BY salary DESC) AS rank FROM employees e)
WHERE rank <= 3;

```

Output:

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID	RANK
1	200	Jennifer	Whalen	JWHALEN	515.123.4444	17-SEP-03	AD_ASST	4400	(null)	101	10	1
2	201	Michael	Hartstein	MHARTSTE	515.123.5555	17-FEB-04	MK_MGR	13000	(null)	100	20	1
3	202	Pat	Fay	PFAY	603.123.6666	17-AUG-05	MK_REP	6000	(null)	201	20	2
4	114	Den	Raphaely	DRAPHEAL	515.127.4561	07-DEC-02	PU_MAN	11000	(null)	100	30	1
5	115	Alexander	Khoo	AKHOO	515.127.4562	18-MAY-03	PU_CLERK	3100	(null)	114	30	2
6	116	Shelli	Baida	SBAIDA	515.127.4563	24-DEC-05	PU_CLERK	2900	(null)	114	30	3
7	203	Susan	Mavris	SMAVRIS	515.123.7777	07-JUN-02	HR_REP	6500	(null)	101	40	1
8	121	Adam	Fripp	AFRIPP	650.123.2234	10-APR-05	ST_MAN	8200	(null)	100	50	1
9	120	Matthew	Weiss	MWEISS	650.123.1234	18-JUL-04	ST_MAN	8000	(null)	100	50	2
10	125	Julia	Nayer	JNAYER	650.124.1214	16-JUL-05	ST_CLERK	3200	(null)	120	50	3
11	103	Alexander	Hunold	AHUNOLD	590.423.4567	03-JAN-06	IT_PROG	9000	(null)	102	60	1

Concepts Used:

Window Function • DENSE_RANK() • PARTITION BY • Top-N Analysis

Query 06

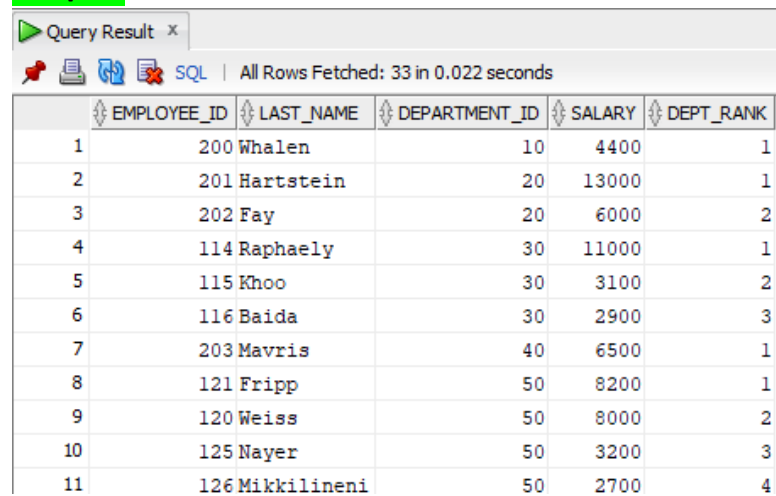
Question:

Show employee salary ranking within each department.

SQL Query:

```
SELECT employee_id, last_name, department_id, salary,  
RANK() OVER (PARTITION BY department_id  
ORDER BY salary DESC) AS dept_rank  
FROM employees;
```

Output:



The screenshot shows a SQL query result window titled 'Query Result x'. It displays 11 rows of data from the 'employees' table, ordered by department_id and then by salary in descending order. The columns are EMPLOYEE_ID, LAST_NAME, DEPARTMENT_ID, SALARY, and DEPT_RANK. The data is as follows:

EMPLOYEE_ID	LAST_NAME	DEPARTMENT_ID	SALARY	DEPT_RANK
1	200 Whalen	10	4400	1
2	201 Hartstein	20	13000	1
3	202 Fay	20	6000	2
4	114 Raphaely	30	11000	1
5	115 Khoo	30	3100	2
6	116 Baida	30	2900	3
7	203 Mavris	40	6500	1
8	121 Fripp	50	8200	1
9	120 Weiss	50	8000	2
10	125 Nayer	50	3200	3
11	126 Mikilineni	50	2700	4

Concepts Used:

RANK() • Window Function • PARTITION BY • ORDER BY

Query 07

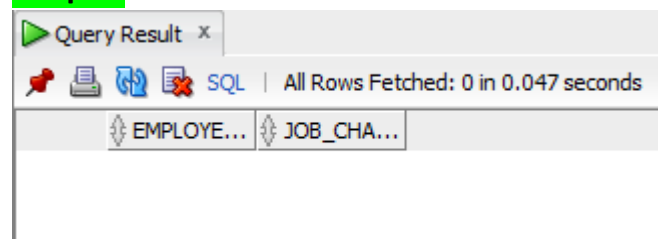
Question:

Find employees who have changed jobs more than once.

SQL Query:

```
SELECT employee_id,  
COUNT(*) AS job_changes  
FROM job_history  
GROUP BY employee_id  
HAVING COUNT(*) > 1;
```

Output:



The screenshot shows a SQL query result window titled 'Query Result x'. It displays the results of a query on the 'job_history' table. The columns are EMPLOYEE_ID and JOB_CHANGES. The results are as follows:

EMPLOYEE_ID	JOB_CHANGES
-------------	-------------

Concepts Used:

GROUP BY • COUNT() • HAVING • Job History Analysis

Query 08

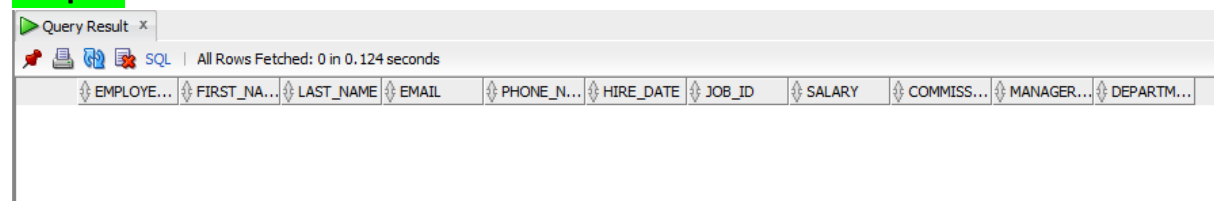
Question:

Find employees whose current salary is outside the defined salary range for their job.

SQL Query:

```
SELECT e.*  
FROM employees e  
JOIN jobs j  
ON e.job_id = j.job_id  
WHERE e.salary < j.min_salary  
OR e.salary > j.max_salary;
```

Output:



EMPLOYEE...	FIRST_NA...	LAST_NAME	EMAIL	PHONE_N...	HIRE_DATE	JOB_ID	SALARY	COMMISS...	MANAGER...	DEPARTM...
-------------	-------------	-----------	-------	------------	-----------	--------	--------	------------	------------	------------

Concepts Used:

JOIN • Business Rule Validation • OR Operator • Data Quality Analysis

Query 09

Question:

Find employees with inconsistent department history.

SQL Query:

```
SELECT DISTINCT e.employee_id  
FROM employees e  
JOIN job_history jh  
ON e.employee_id = jh.employee_id  
WHERE e.department_id != jh.department_id;
```

Output:

[Insert SQL Developer screenshot here]

Concepts Used:

JOIN • DISTINCT • Data Validation • Historical Data Analysis

Query 10

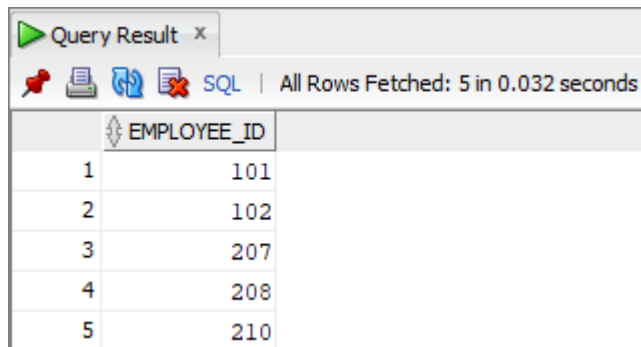
Question:

Show employees in the top 20% by salary within each department.

SQL Query:

```
SELECT *FROM (SELECT e.*, PERCENT_RANK() OVER (
PARTITION BY department_id
ORDER BY salary DESC) AS salary_percent_rank
FROM employees e)
WHERE salary_percent_rank <= 0.20
ORDER BY department_id, salary DESC;
```

Output:



The screenshot shows a 'Query Result' window with a toolbar containing icons for a pin, print, refresh, and SQL editor. The status bar indicates 'All Rows Fetched: 5 in 0.032 seconds'. The table has two columns: 'EMPLOYEE_ID' and an unlabeled column. The data is as follows:

	EMPLOYEE_ID	
1	101	
2	102	
3	207	
4	208	
5	210	

Concepts Used:

PERCENT_RANK() • Window Function • PARTITION BY • Percentile Analysis •
Advanced SQL